



Customer Churn analysis

Presentata da:

Pietro Balestri, Marco Fronzoni Patti

Terzo appello estivo

Anno Accademico 2025/2026

Indice

1	Introduzione	2
1.1	Obiettivi del progetto	3
2	Analisi Preliminare dei dati	4
2.1	Presentazione del dataset	4
2.2	Osservazioni	5
2.3	Pulizia e preparazione dei dati	6
2.4	Dataset con classi sbilanciate	9
2.5	Preprocessing	10
3	Machine Learning	12
3.1	Stochastic Gradient Descent	12
3.2	K Nearest Neighbours	14
3.3	Logistic Regression	16
3.4	Conclusioni	17
4	Web Server	19
4.1	Implementazione del servizio web	19
4.2	Git	19
4.3	Docker	19
4.4	Utilizzo	20

Capitolo 1

Introduzione

L'analisi dell'abbandono dei clienti ("Churn") riveste un ruolo rilevante dal punto di vista economico, poiché consente a un'azienda di comprendere le cause della perdita di clienti, stimarne l'impatto economico e definire strategie volte alla sua riduzione. In particolare, nel settore delle telecomunicazioni i clienti possono cambiare frequentemente operatore; per questo motivo, il mantenimento della clientela esistente risulta spesso più conveniente rispetto all'investimento necessario per acquisire nuovi clienti attraverso campagne di marketing e promozione. La presente relazione descrive quindi l'analisi del dataset *Telco Customer Churn* di IBM¹, comprendente le fasi di esplorazione e preparazione dei dati, l'addestramento e l'ottimizzazione degli iperparametri di tre differenti modelli di machine learning, il confronto delle loro prestazioni e l'implementazione di un'applicazione web per il deployment dei modelli tramite container Docker. Il codice per l'analisi dei dati, l'addestramento, e l'applicazione web è stato tenuto sotto version control in una repository di github, al seguente link: <https://github.com/balpie/CustomerChurnAnalysis>

¹Disponibile su Kaggle: <https://www.kaggle.com/datasets/blanchard/telco-customer-churn>

1.1 Obiettivi del progetto

- Sviluppare modelli con tecniche di machine learning per predire eventuali abbandoni di servizi telco da parte di utenti.
- Confrontare le performance dei modelli allenati e valutare i migliori.

Capitolo 2

Analisi Preliminare dei dati

L'analisi preliminare dei dati è stata necessaria per comprendere la struttura del dataset, individuare eventuali problemi nei dati ed effettuare la fase di preprocessing.

2.1 Presentazione del dataset

Il dataset su cui abbiamo lavorato raccoglie informazioni relative ai sottoscrittori di uno o più servizi di telecomunicazione (ad es. *PhoneService*, *InternetService*, *StreamingMovies*, ecc.), con lo scopo di prevenire l'abbandono della sottoscrizione a tali servizi.

Osservando il dataset abbiamo potuto notare la presenza di alcune feature categoriche legate ad altre feature. Esse rappresentano servizi aggiuntivi collegati ad un altro servizio. Tra queste feature un esempio è *MultipleLines*, la quale distingue gli utenti tra coloro che possiedono più linee telefoniche, coloro che possiedono un'unica linea telefonica o coloro che non hanno una linea telefonica. Proprio per il suo significato la feature *MultipleLines* è legata alla feature *PhoneService*, la quale indica se l'utente possiede o non possiede una linea telefonica. Questo non è l'unico caso e ulteriori casi sono osservabili nella matrice di correlazione (Figura 2.1).

Tabella 2.1: Descrizione delle feature del dataset Telco .

Nome Feature	Descrizione
customerID	ID univoco assegnato a ciascun cliente.
gender	Genere del cliente (Maschio o Femmina).
SeniorCitizen	Indica se il cliente è un anziano (1 = Sì, 0 = No).
Partner	Indica se il cliente ha un partner (Sì o No).
Dependents	Indica se il cliente ha persone a carico (Sì o No).
tenure	Numero di mesi in cui il cliente è rimasto abbonato all'azienda.
PhoneService	Indica se il cliente ha un servizio telefonico attivo (Sì o No).
MultipleLines	Indica se il cliente ha linee telefoniche multiple (Yes, No, No phone service).
InternetService	Tipo di provider per il servizio internet (DSL, Fibra ottica, No).
OnlineSecurity	Indica se il cliente ha attivato il servizio di sicurezza online.
OnlineBackup	Indica se il cliente ha un servizio di backup online.
DeviceProtection	Indica se il cliente ha un piano di protezione per il dispositivo.
TechSupport	Indica se il cliente ha un piano di supporto tecnico.
StreamingTV	Indica se il cliente utilizza il servizio di streaming TV.
StreamingMovies	Indica se il cliente utilizza il servizio di streaming di film.
Contract	Tipologia di contratto stipulato (Mensile, Annuale, Biennale).
PaperlessBilling	Indica se il cliente ha scelto la fatturazione elettronica (Sì o No).
PaymentMethod	Il metodo di pagamento utilizzato dal cliente.
MonthlyCharges	L'importo mensile addebitato al cliente.
TotalCharges	L'importo totale addebitato al cliente fino a quel momento.
Churn (Target)	Indica se il cliente ha abbandonato il servizio nel mese precedente (Sì o No).

2.2 Osservazioni

Abbiamo osservato che alcune delle feature sono più correlate di altre per quanto riguarda l'abbandono dei servizi, come per esempio il metodo di pagamento: gli utenti che pagano

mediante assegno elettronico abbandonano più spesso i servizi, anche in proporzione. Inoltre tale metodo di pagamento è il più comune per gli utenti [2.2]. E' stata calcolata anche la matrice delle correlazioni [2.1], applicando One Hot encoding [2.5] per evitare che l'encoding delle feature di tipo categorico influisse sul risultato. Altre feature sono state considerate meno significative, come per esempio il genere [2.3]. Nonostante queste considerazioni, alla luce del costo molto basso dell'allenamento dei modelli scelti, è stata presa la decisione di mantenere tutte le feature del modello.

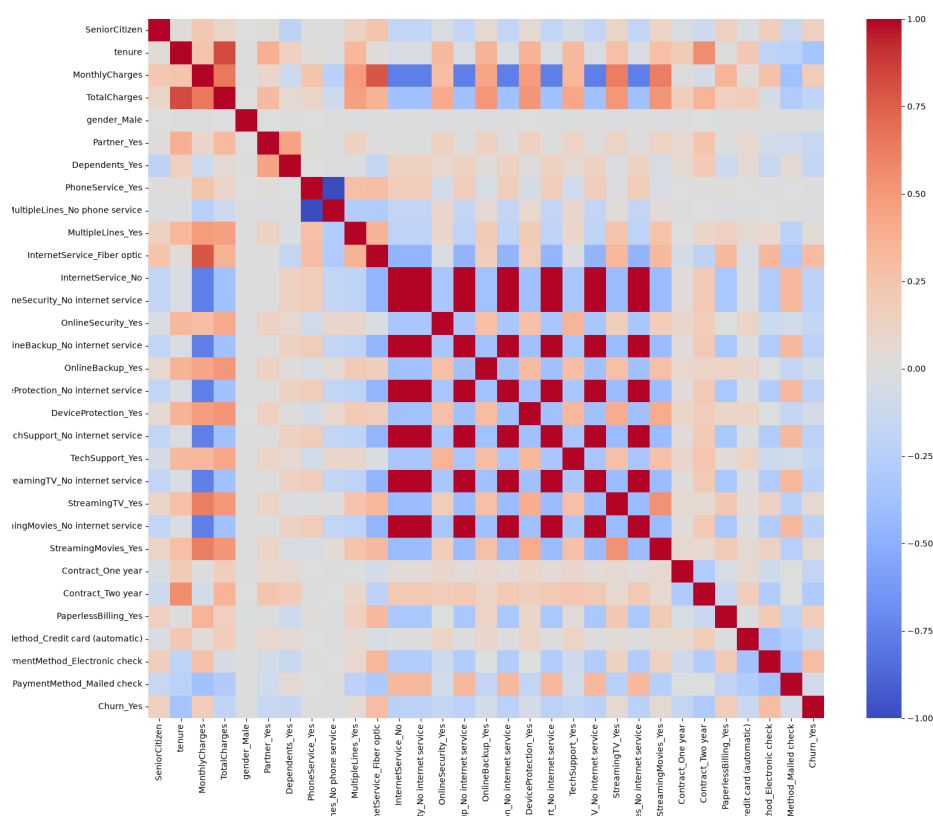


Figura 2.1: matrice delle correlazioni

2.3 Pulizia e preparazione dei dati

Il dataset è un dataset denso e contiene 7043 record. L'uniche pulizie che sono state necessarie sono state nei confronti della feature *TotalCharges*. Feature la quale, nonostante indicasse un

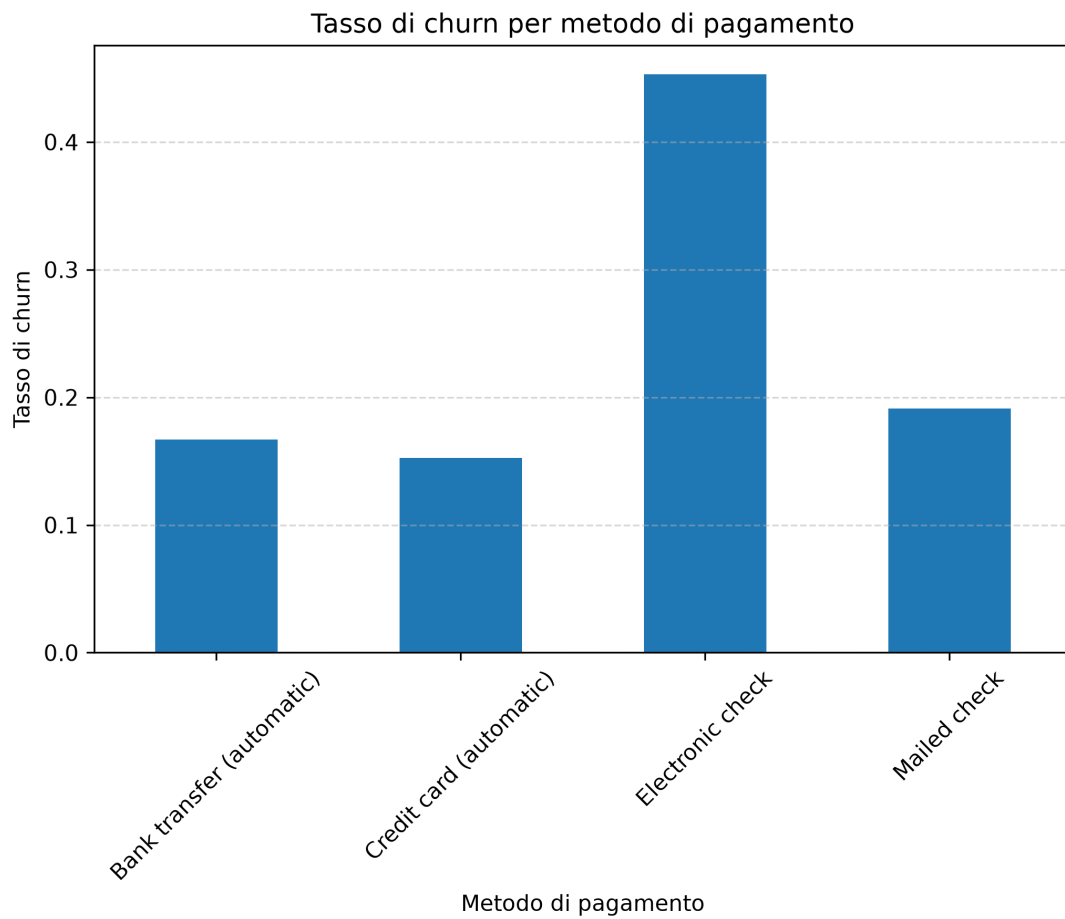


Figura 2.2: Grafico sulla distribuzione dell'abbandono dei clienti per metodo di pagamento

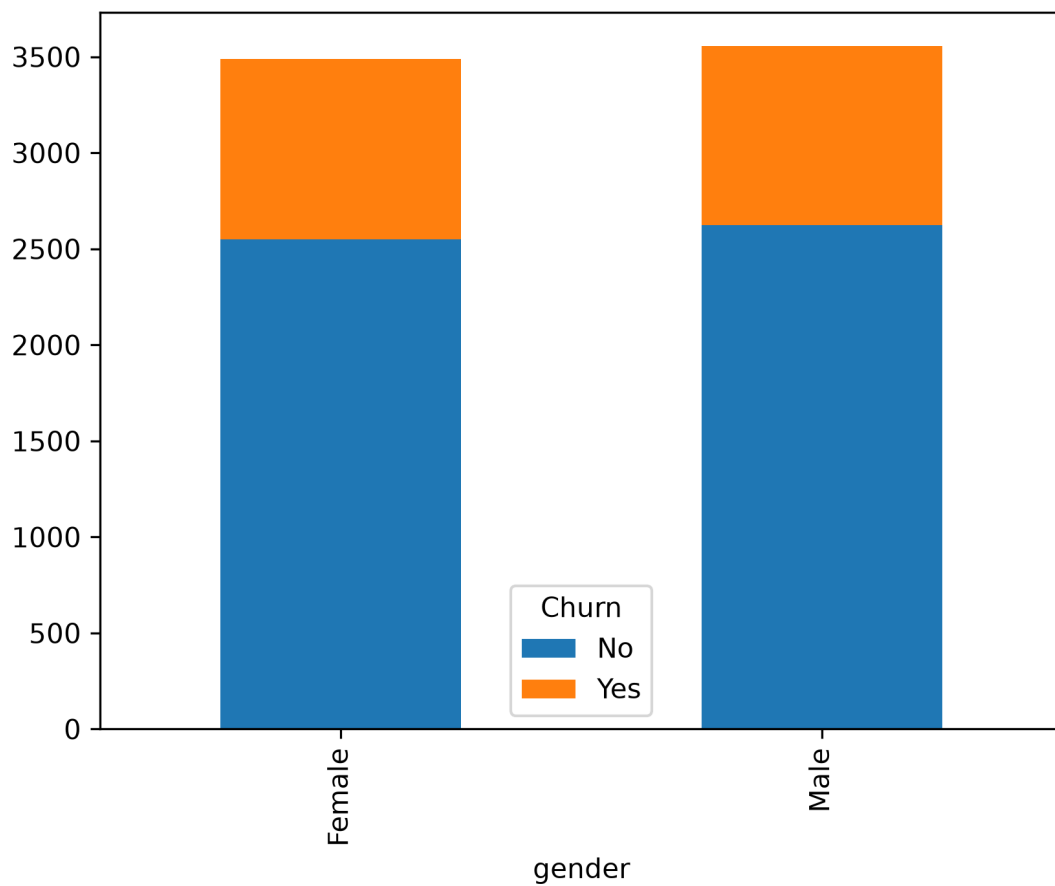


Figura 2.3: Grafico sulla distribuzione dell'abbandono dei clienti per genere

valore numerico, si presentava come una stringa. Investigando sui dati, abbiamo scoperto che il motivo per cui si presentava in tale modo era dovuto alla presenza di stringhe vuote che si presentavano esclusivamente in corrispondenza di utenti iscritti da meno di un mese (*tenure* = 0). Ciò è stato interpretato da noi come indicazione del fatto che non sono stati ancora effettuati pagamenti da parte di tali utenti e abbiamo sostituito tale valore di stringa vuota con il valore numerico zero.

customerID	tenure	TotalCharges
4472-LVYGI	0	
3115-CZMZD	0	
5709-LVOEQ	0	
4367-NUYAO	0	
1371-DWPAZ	0	
7644-OMVMY	0	
3213-VVOLG	0	
2520-SGTTA	0	
2923-ARZLG	0	
4075-WKNIU	0	
2775-SEFEE	0	

Tabella 2.2: Dettaglio dei clienti con valori mancanti nei costi totali, tutti con *tenure* uguale a zero.

2.4 Dataset con classi sbilanciate

Fin da subito abbiamo osservato che la quantità di utenti che hanno abbandonato la sottoscrizione ai servizi (26,5%) è considerevolmente inferiore alla quantità di utenti ancora iscritti ai servizi (73,5%) (Vedi Figura 3.2). Notare subito questa distribuzione sbilanciata nelle classi target ci ha permesso di riprodurre tale distribuzione nella suddivisione tra training set e test set in modo da avere una suddivisione rappresentativa della realtà e ci ha permesso di effet-

tuare una migliore scelta delle metriche da tenere maggiormente in considerazione, in quanto l'accuratezza è fuorviante in presenza di classi sbilanciate. *(Per comprendere meglio questa affermazione basta considerare il problema di classificare se un numero è un numero primo o no. Un classificatore che dice sempre che il numero non è un numero primo, è molto accurato poiché i numeri non primi sono più frequenti dei numeri primi. Tuttavia è un pessimo classificatore poiché nessun numero primo viene classificato come tale).*

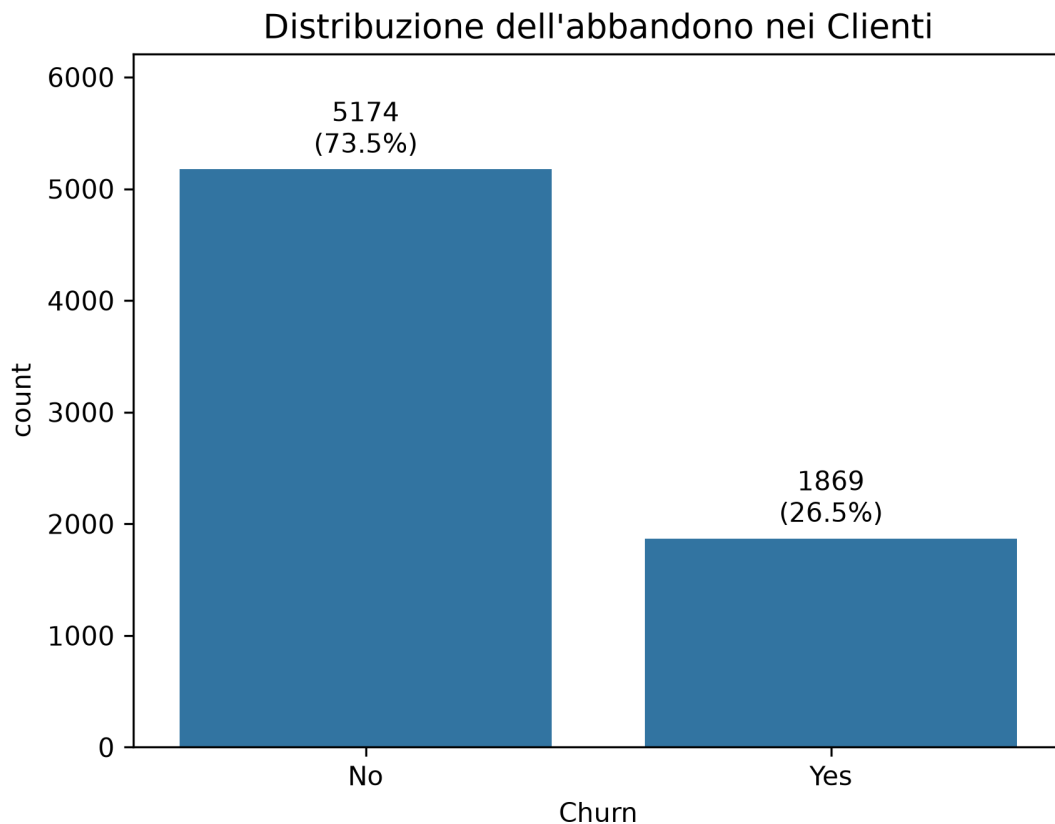


Figura 2.4: Grafico sulla distribuzione dell'abbandono dei clienti

2.5 Preprocessing

Prima di poter partire con l'addestramento è stato necessario effettuare una fase di preprocessing, articolata in due operazioni:

One Hot Encoding: Encoding che permette di trasformare le feature categoriche in un insieme di variabili binarie. L'encoding di questo tipo è necessario perché i modelli possono essere addestrati solo su dati numerici e non su dati categorici.

MultipleLines	<i>cat__MultipleLines_No</i>	<i>cat__MultipleLines_No phone service</i>	<i>cat__MultipleLines_Yes</i>
No	1	0	0
No phone service	0	1	0
Yes	0	0	1

Tabella 2.3: Esempio di codifica One Hot per la feature *MultipleLines*.

Standard Scaler: Scala tutte le feature numeriche, trasformando i dati in modo che abbiano media pari a 0 e deviazione standard pari a 1. Ciò è importante per garantire che le feature numeriche abbiano tutte lo stesso peso e che il risultato non dipenda maggiormente da feature con valori numerici alti (Tabella 2.4).

	<i>tenure</i>	<i>MonthlyCharges</i>	<i>TotalCharges</i>
min	0.00	18.25	0.00
max	72.00	118.75	8684.80
media	32.37	64.76	2279.73

Tabella 2.4: Minimo, massimo e media delle feature con valori numerici prima dell'operazione di Standard Scaler.

Capitolo 3

Machine Learning

Per quanto riguarda la fase di addestramento, abbiamo preso in considerazione tre diverse tipologie di modello: Stochastic Gradient Descent, K Nearest Neighbours e Logistic Regression. Per ognuna delle tre tipologie abbiamo effettuato una ricerca empirica al fine di individuare la combinazione ottimale di iperparametri con le performance più elevate possibile. Tale ricerca è stata effettuata tramite la funzione `GridSearchCV`, la quale, fornendole una lista di iperparametri e un insieme finito di valori per ogni iperparametro, prova tutte le combinazioni possibili e effettuando `CrossValidation`, restituisce una classifica delle migliori combinazioni di iperparametri su una performance a scelta. Infine abbiamo confrontato le performance delle migliori combinazioni di iperparametri tra le 3 tipologie di modello scelto e tratto le nostre conclusioni.

3.1 Stochastic Gradient Descent

Per il modello *Stochastic Gradient Descent*, sono stati scelti da esplorare i seguenti iperparametri:

- **loss**: la loss function utilizzata, con opzioni **hinge**, **log_loss**, **modified_huber**
- **alpha**: costante moltiplicativa della penalità (penalty), con valori esplorati di $[1e-5, 1e-4, 1e-3, 1e-2]$.
- **penalty**: il tipo di regolarizzazione, con opzioni *L1* (Lasso), *L2* (Ridge) e *elasticnet* il quale calcola un mix tra le prime due, dove per quest'ultima è stato specificato anche:

- **l1_ratio**: il parametro di miscelazione utilizzato nella formula della *elasticnet*, con valori esplorati di [0.15, 0.5, 0.85]

Successivamente è stato introdotto nella ricerca anche l'utilizzo del parametro **class_weight**, che precedentemente era impostato uguale a *None*. Impostando **class_weight** = "balanced" le performance hanno subito un sensibile miglioramento, come riportato dal grafico Figura 3.1.

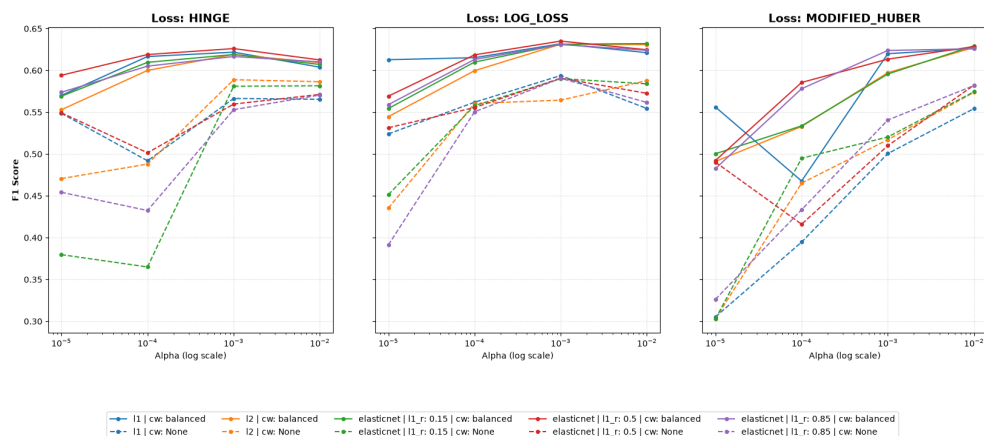


Figura 3.1: Risultati grid search per SGD, divisi nei tre grafici per funzione di loss, ogni singola traccia rappresenta una penalty al variare di alpha. In tratteggiato le tracce che non tengono conto di **class_weight**.

Parametro	Valore
alpha	0.001
class_weight	balanced
loss	log_loss
penalty	elasticnet
l1_ratio	0.5

Tabella 3.1: Migliori parametri ottenuti dalla Grid Search

3.2 K Nearest Neighbours

Per il modello *K Nearest Neighbours*, la metrica *Minkowski*, tiene conto del parametro **p**, mentre le metriche standard no:

- Per le metriche **euclidean** e **manhattan**:
 - **n_neighbours**: il numero di vicini da considerare, con valori nell'intervallo $[2; k+10]$, dove **k** è $\sqrt{n}$, con n numero di record nel training set utilizzato.
 - **weights**: la funzione peso utilizzata nella previsione, con opzioni *uniform* (tutti i punti nel vicinato hanno lo stesso peso) e *distance* (il peso dei punti è inversamente proporzionale alla loro distanza).
- Per la metrica **minkowski**:
 - **n_neighbours**: il numero di vicini da considerare, con valori nell'intervallo $[2; k+10]$.
 - **weights**: la funzione peso, con opzioni *uniform* e *distance*.
 - **p**: il parametro di potenza per la metrica di Minkowski, con valori esplorati di $[1.5, 3, 4]$.

Parametro	Valore
metric	manhattan
n_neighbors	37
weights	uniform
p	NaN (non applicabile)

Tabella 3.2: Migliori parametri ottenuti dalla Grid Search per KNN



Figura 3.2: Performance KNN al variare di k

3.3 Logistic Regression

Per il modello di *Logistic Regression*, la ricerca degli iperparametri variava a seconda del solutore scelto, in quanto non tutti i solutori accettavano tutti i tipi di penalty: [3.3] [3.3]

- Per il solutore `liblinear`:

- **l1_ratio**: il tipo di regolarizzazione o parametro di miscelazione, con opzioni 0 (*l2* o Ridge) e 1 (*l1* o Lasso).
- C: l'inverso della forza di regolarizzazione, con valori esplorati su scala logaritmica di [0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000, 10000]. Valori più piccoli di C implicano una regolarizzazione più forte.
- **class_weight**: il bilanciamento delle classi, con opzioni *balanced* (pesi adattati inversamente alla frequenza delle classi) e *None* (nessun peso).

- Per il solutore `lbfgs`:

- **l1_ratio**: solo l'opzione 0 (*l2*) è stata considerata, in quanto `lbfgs` non supporta la regolarizzazione *l1*.
- C: l'inverso della forza di regolarizzazione, con gli stessi valori esplorati di [0.0001, 0.001, 0.01, 0.1,
- **class_weight**: il bilanciamento delle classi, con opzioni *balanced* e *None*.

- Per il solutore `saga`:

- **l1_ratio**: il tipo di regolarizzazione, con opzioni 0 (*l2* o Ridge), 1 (*l1* o Lasso) e valori intermedi di [0.3, 0.5, 0.7] che corrispondono a una regolarizzazione di tipo *elasticnet*.
- C: l'inverso della forza di regolarizzazione, con gli stessi valori esplorati di [0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000, 10000].
- **class_weight**: il bilanciamento delle classi, con opzioni *balanced* e *None*.

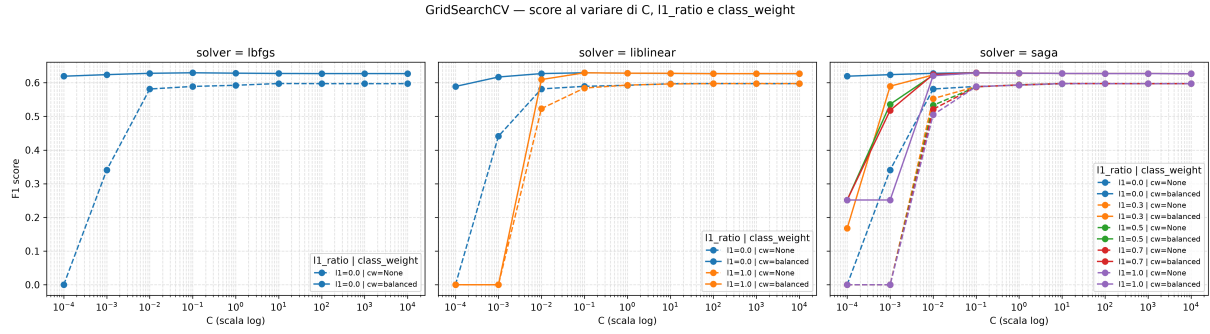


Figura 3.3: Performance Logistic regression al variare di C

Parametro	Valore
C	0.1
class_weight	balanced
l1_ratio	0.0
solver	saga

Tabella 3.3: Migliori parametri ottenuti dalla Grid Search Per LogisticRegression

3.4 Conclusioni

In conclusione, confrontando le performance dei diversi modelli (Figura 3.5 , Figura 3.4), abbiamo osservato risultati in generale leggermente migliori nei modelli basati su Stochastic Gradient Descent e Logistic Regression piuttosto che nel modello K Nearest Neighbour. Anche se il punteggio f1 dei tre modelli differisce di poco, data la natura del problema affrontato, è stata attribuita maggiore importanza alla recall. Tale scelta deriva dall'assunzione che il costo associato a un falso negativo sia superiore rispetto a quello associato a un falso positivo: non identificare un cliente intenzionato ad abbandonare può comportare la perdita definitiva dello stesso, mentre classificare erroneamente un cliente come churner può comportare un costo operativo legato a un'azione di retention non necessaria.

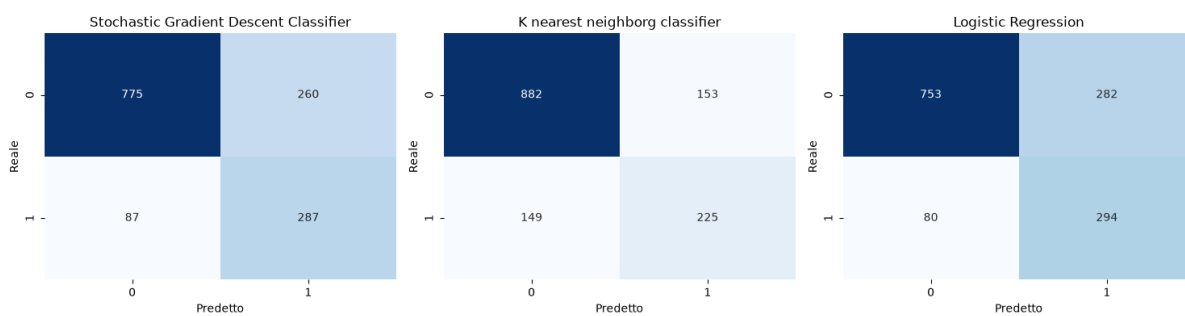


Figura 3.4: Matrice di confusione dei tre modelli

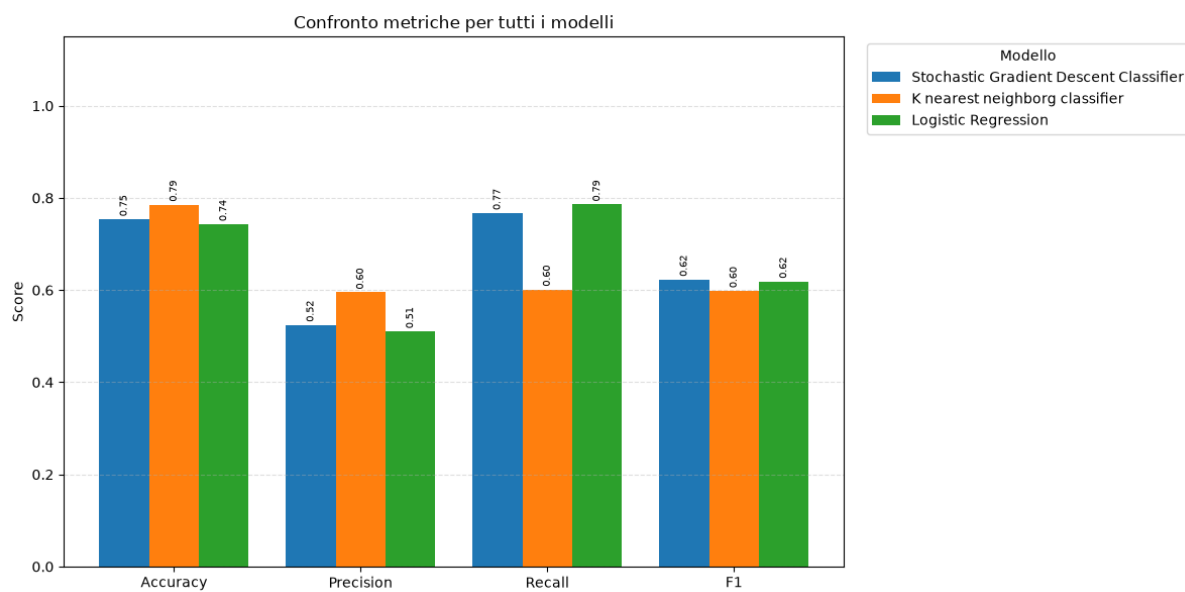


Figura 3.5: Confronto Metriche

Capitolo 4

Web Server

4.1 Implementazione del servizio web

Per rendere i modelli di machine learning utilizzabili tramite un'interfaccia grafica è stato sviluppato un servizio web basato sul framework Flask. L'obiettivo del web server è quello di fornire un punto di accesso all'inferenza dei modelli, permettendo all'utente di inserire i dati relativi a un cliente e ottenere una previsione sull'eventuale abbandono del servizio.

4.2 Git

Per ogni feature aggiunta è stato creato un branch su GitHub, del quale poi è stato fatto il merge con il main branch. Questo ha permesso di collaborare nello sviluppo del codice parallelamente, mantenendo sempre una versione stabile nel branch principale.

4.3 Docker

L'applicazione è distribuita mediante Docker, così da garantire un ambiente di esecuzione indipendente dal sistema operativo e dalle librerie installate localmente. Inoltre, l'utilizzo di container facilita una futura evoluzione dell'architettura verso una suddivisione in più servizi indipendenti, qualora fosse necessario separare componenti quali l'interfaccia utente, il backend e il servizio di inferenza.

4.4 Utilizzo

Per eseguire l'applicazione è necessario avere installati `Git` e `Docker`. Per prima cosa è necessario clonare la repository del progetto mediante il comando:

```
git clone https://github.com/balpie/CustomerChurnAnalysis.git
```

Successivamente, dalla directory principale del progetto, è possibile costruire l'immagine `Docker` eseguendo:

```
docker build -t appflask .
```

Una volta completata la build, il container può essere avviato tramite:

```
docker run -p 5000:5000 appflask
```

L'applicazione sarà quindi accessibile all'indirizzo `http://localhost:5000` dal browser.